

Diffusion GAN-Based Oversampling for Imbalanced Tabular Data

Shiqi Ren^{ID}, Jinliang Ding^{ID}, *Senior Member, IEEE*, and Yiu-ming Cheung^{ID}, *Fellow, IEEE*

Abstract—Imbalanced class distribution disrupts the training of a classifier, resulting in biases favoring majority classes. Data oversampling is a common strategy to tackle this issue. However, traditional methods may generate incorrect and unnecessary instances when facing complex data challenges, such as class overlap, small disjuncts, and noise samples. Therefore, there is a need for an oversampling method that can accurately characterize the data distribution. This paper introduces a novel deep generative oversampling approach for balancing the imbalanced tabular data by leveraging diffusion models and Generative Adversarial Networks (GANs). The model comprises a generator constructed from diffusion models and a discriminator with a Noise-Sensitive Auxiliary Classifier (NSAC) and is trained through an adversarial process. The synergy of these two models enhances stability and sample quality compared to GANs, with faster sampling speed and better conditional generating ability than diffusion models. In experimental validation across 22 real-world datasets, our method consistently outperforms six counterparts regarding Accuracy, F1-score, and MCC for binary and multi-class scenarios. Notably, our approach enhances classifier accuracy for minority classes while maintaining a high level for the majority class, a facet often compromised by other algorithms.

Index Terms—Imbalanced data, tabular data, diffusion models, generative adversarial networks, oversampling.

I. INTRODUCTION

IMBALANCED classification of tabular data is widespread in real-world applications, including fraud detection [1], medical diagnosis [2], and fault analysis [3]. Tabular data is ubiquitous in data mining and machine learning [4], where each row represents a sample with a fixed set of features. Class imbalance refers to skewed distributions among classes, with one or more classes having significantly fewer instances.

Received 15 January 2024; revised 6 July 2024; accepted 24 November 2025. Date of publication 2 December 2025; date of current version 30 December 2025. The work was supported in part by the Xingliao Talent Program of Liaoning Province under Grant XLYC2202002, in part by NSFC/Research Grants Council (RGC) Joint Research Scheme N under Grant HKBU214/21, in part by the General Research Fund of under Grant RGC 12201323 and Grant 12202025, and in part by RGC Senior Research Fellow Scheme under Grant SRFS2324-2S02. Recommended for acceptance by J. Tang. (Corresponding author: Jinliang Ding.)

Shiqi Ren and Jinliang Ding are with the State Key Laboratory of Synthetical Automation for Process Industries, Northeastern University, Shenyang 110819, China (e-mail: rsq.hkbat@gmail.com; jlding@mail.neu.edu.cn).

Yiu-ming Cheung is with the Department of Computer Science, Hong Kong Baptist University, Hong Kong, SAR 999077, China (e-mail: ymc@comp.hkbu.edu.hk).

This article has supplementary downloadable material available at <https://doi.org/10.1109/TKDE.2025.3639433>, provided by the authors.

Digital Object Identifier 10.1109/TKDE.2025.3639433

The skewness in the underlying data distribution can seriously undermine the effectiveness of machine learning methods, particularly for minority classes, because the classifier tends to ignore them and predict all instances to the majority class. Therefore, imbalanced data learning aims to develop a classifier that achieves high accuracy for minority classes while maintaining a good level of accuracy for the majority class [5].

Over the past years, imbalanced learning methods can be broadly categorized into two groups [6]: the algorithm-level methods and the data-level methods. The algorithmic approaches such as cost-sensitive methods [7] and ensemble learning algorithms [8] are among the proposed approaches for addressing these challenges. Cost-sensitive methods enforce the accurate classification of minority class samples by assigning a high cost to their misclassification. However, cost-sensitive methods require prior knowledge of the practical scenarios and may lead to overfitting in training [9]. Ensemble learning applies bagging [10] and boosting [11] algorithms to enhance the performance of a learning method by combining a group of base classifiers within the classification system [6]. However, ensemble learning is not originally designed for class imbalance problems and is often combined with other imbalanced learning methods [12].

On the other hand, the data-level approaches, including under-sampling [13], [14] and oversampling methods [15], [16], [17], employs resampling techniques to rebalance the data distribution. The under-sampling methods eliminate some majority class instances until a balanced distribution is achieved. Besides, oversampling techniques increase the number of minority class instances to reach the desired level of balance. While oversampling methods may entail longer training times than under-sampling, they often yield superior results when addressing severely imbalanced data [18]. This research focuses on oversampling methods that can be integrated with various machine learning methods and maximize the utilization of original data information.

An imbalanced dataset often accompanies additional complications in real-world classification tasks, including class overlap, small disjuncts, and noise examples. Classical statistical oversampling techniques such as Synthetic Minority Oversampling Technique (SMOTE) [15] and various SMOTE variants have witnessed substantial advancements and widespread utilization [19], [20]. However, these techniques cannot effectively tackle data complexities and can magnify the degree of complexities [21]. These continuously emerging challenges

keep the field expanding, calling for novel and effective solutions to analyze, understand, and tackle these data-level difficulties. Deep Generative Models (DGM) [22], [23], [24], featuring with the powerful ability to characterize the target data distribution, are promising for oversampling. In the literature, a number of deep generative oversampling methods [25], [26], [27] have been recently proposed to address the data imbalance problem.

As the most prevalent deep generative model (DGM)-based oversampling methods, Generative Adversarial Networks [23] (GAN)-based approaches balance skewed data distribution by generating minority data through a one-shot mapping from latent space to data distribution. However, GAN-based models often encounter challenges like limited sample fidelity and mode collapse. A recent breakthrough in deep generative models is diffusion models [24], [28], [29], which generate examples through a gradual sampling process from noise after learning to characterize an explicit likelihood. Compared with the one-shot generative process of GANs, the diffusion models have an advantage in generation quality and mode coverage, but poor sampling speed.

In this paper, we will propose a **Diffusion GAN-based Oversampling** approach for imbalanced **Tabular data (DGOT)**, designed to balance the skewed data distribution by leveraging diffusion models and GANs. DGOT consists of a conditional diffusive generator based on diffusion models and a discriminator with a Noise-Sensitive Auxiliary Classifier (NSAC). The generator synthesizes the minority class data to address the imbalanced class problem after training through an adversarial learning process. The synergy of these two models enhances stability and sample quality compared to GANs, with faster sampling speed and greater conditional generating ability than diffusion models.

Differing from the generator of the vanilla GANs, our model breaks down the one-shot generation process into several simpler denoising diffusion steps. This step-wise approach smoothes data distribution through a diffusion process, enhancing stability during adversarial training. Moreover, unlike regular diffusion models, the adversarial training process empowers DGOT to adopt a swift diffusion process with larger steps, thereby improving efficiency. Additionally, by investigating the distinctions between different classes of tabular data under varying noise levels in the diffusion process by a Conditional Probability Distance (CPD) metric, we introduce a discriminator with a noise-sensitive auxiliary classifier (NSAC). This unique discriminator encourages the generator to prioritize learning features that are challenging to distinguish at the high noise level and also guides the generator in synthesizing data for each class conditionally, particularly crucial for minority classes.

Extensive experiments are conducted using various oversampling methods on 22 real-world datasets. Experimental results indicate the efficacy of the proposed method on addressing the imbalanced tabular data. The main contributions of this paper are summarized as follows:

- We propose the first generative oversampling approach based on the diffusion models and GANs to generate

minority examples for imbalanced tabular data problems conditionally.

- We design the first diffusion-based generator of GANs for high-fidelity tabular data conditional synthesis.
- We explore feature variations in tabular data from different classes at varying noise levels using a novel CPD metric.
- We introduce a novel NSAC structure in the discriminator of GANs to improve the ability of the diffusion-based generator.
- We evaluate our approach on 22 real-world datasets compared to six baseline methods. The experimental results demonstrate that our approach achieves superior performance.

The remainder of the paper is organized as follows. Section II makes an overview of the related work. Section III describes the proposed method in detail. The experimental settings and analyses of the results are presented in Section IV. Finally, Section V draws a conclusion.

II. RELATED WORKS

This section makes an overview of the related works. We firstly review the state-of-the-art oversampling approaches. Then, we delve into the definitions of diffusion models and GANs. Finally, we briefly introduce research efforts focused on generative models for tabular data.

A. Oversampling Approaches in Imbalanced Learning

Oversampling approaches are employed to rectify skewed data distributions by augmenting instances of the minority class [30]. As a resampling technique, it is more versatile than other methods that can be seamlessly integrated with various machine learning techniques [17]. The primary methodologies for this approach includes classical statistical methods and DGM-based approaches [27].

SMOTE is the most important technique among the classical statistical oversampling methods that generate synthetic samples by interpolating between a sample and its nearest neighbors within the same class [15]. This efficient and effective sampling method has demonstrated superior results in many tasks and inspired many researchers to develop different synthetic oversampling approaches [16], [31]. However, recent works show that SMOTE-based methods cannot properly deal with multimodal data and cases with high class overlap or noise [21]. Therefore, there is a calling for new oversampling approaches to tackling the imbalanced data.

Deep generative models with a powerful ability to characterize the data distribution show promise in addressing imbalanced data by synthesizing minority class samples. Among these models, oversampling methods based on GANs are particularly popular, with various related works proposed [26], [32]. However, a notable challenge in data synthesis with GANs is mitigating mode collapse arising from the adversarial learning process. Additionally, there are algorithms based on other deep generative models. For instance, DeepSMOTE [25] combines the SMOTE algorithm with a Variational Autoencoder (VAE)

model to rebalance imbalanced image data. Kim et al. [27] introduced a specialized score network called SOS for tabular data oversampling. These studies showcase promising results in the generation of class-imbalanced data.

B. Generative Diffusion Models

Diffusion models define a Markov chain of diffusion steps to gradually add noise to the data and then learn to reverse the diffusion process to construct desired data samples from the noise. This approach can be divided into forward diffusion and reverse denoising processes. Inspired by this theory, two landmark works, DDPM [24] and Score SDE [29], respectively, discuss the discrete and continuous cases. In this paper, our research focuses on discrete-time diffusion models, which can also be easily generalized to continuous-time ones.

During the forward diffusion process, the data distribution $x_0 \sim q(x)$ is gradually added noise in T steps, producing a sequence of noisy samples $x_1, \dots, x_T$. The step sizes are controlled by a variance schedule $\beta(t) \in (0, 1)_{t=1}^T$. Then, the forward process $q(x_{1:T}|x_0)$ can be defined as:

$$q(x_{1:T}|x_0) = \prod_{t=1}^T q(x_t|x_{t-1}), \quad (1)$$

where $q(x_t|x_{t-1})$ refers to the forward step:

$$q(x_t|x_{t-1}) = N(x_t; \sqrt{1 - \beta(t)}x_{t-1}, \beta(t)I), t \in [1, T]. \quad (2)$$

Then, for an arbitrary noisy data $x_t \sim q(x_t|x_0)$, it can be denoted as:

$$\begin{aligned} x_t &= \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon_0 \\ &\sim N(x_t; \sqrt{\bar{\alpha}_t}x_0, \sqrt{1 - \bar{\alpha}_t}I), \end{aligned} \quad (3)$$

where $\alpha = 1 - \beta(t)$ and $\bar{\alpha}_t = \prod_{s=1}^t \alpha_s$. The Signal-to-Noise Rate (SNR) of x_t can be defined as a ratio of squares of mean and variance, which can be written as:

$$SNR(t) = \frac{\bar{\alpha}_t}{1 - \bar{\alpha}_t}, \quad (4)$$

The $SNR(t)$ is a monotonically decreasing function of t . In other words, the distinguishable features of the data samples gradually decrease as the forward diffusion step increases. Eventually, when $t = T$, x_T is equivalent to an isotropic Gaussian distribution as defined in (3).

Same as the forward process, the reverse denoising process also forms a Markov chain to gradually denoise the samples from x_T to x_0 . Note that with a sufficiently small variance schedule $\beta(t)$, the reverse step will also follow a Gaussian distribution with the mean $\mu(x_t, t)$ and variance $\Sigma(x_t, t)$ with the mean $\mu(x_t, t)$ and variance $\Sigma(x_t, t)$ [28]:

$$q(x_{t-1}|x_t) = N(x_{t-1}; \mu(x_t, t), \Sigma(x_t, t)). \quad (5)$$

Then, the reverse denoising process can recreate the true samples x_0 from a Gaussian noise input $x_T \sim N(0, I)$ by learn a model p_θ to approximate the reverse step $q(x_{t-1}|x_t)$. The reverse

process can be defined as follow:

$$p(x_{0:T}) = p(x_T) \prod_{t=1}^T p_\theta(x_{t-1}|x_t), \quad (6)$$

and the model p_θ can be parameterized as:

$$p_\theta(x_{t-1}|x_t) = N(x_{t-1}; \mu_\theta(x_t, t), \Sigma_\theta(x_t, t)), t \in [1, T], \quad (7)$$

where μ_θ and Σ_θ are the mean and variance for the reverse Gaussian transitions parameterized by θ . As the result, the data distribution can be represented as the following joint distribution:

$$p(x_0) = \int p_\theta(x_{0:T}) dx_{1:T}. \quad (8)$$

The model training is performed by minimize a variational bound on the negative log-likelihood of $p(x_0)$. Then, the objective can be rewritten as following:

$$\begin{aligned} L = \mathbb{E}_q \left[\underbrace{D_{KL}(q(x_T|x_0)||p(x_T))}_{L_T} \right. \\ \left. + \sum_{t=1}^{T-1} \underbrace{[D_{KL}(q(x_t|x_{t+1})||p_\theta(x_t|x_{t+1}))]}_{L_{t-1}} \right. \\ \left. - \underbrace{\log p_\theta(x_0|x_1)}_{L_0} \right], \end{aligned} \quad (9)$$

where D_{KL} denotes the Kullback-Leibler (KL) divergence. L_T is a constant depending on variance schedule and can be ignored during training. L_0 can be interpreted as reconstruction loss. L_{t-1} is a combination of several KL divergences between $q(x_{t-1}|x_t)$ and $p_\theta(x_{t-1}|x_t)$. Each term in L_{t-1} compares two Gaussian distributions and therefore they can be computed in a close form.

As mentioned above, the reverse distribution follows Gaussian assumption by imposing a strict infinitesimal limit of variance schedule. Therefore, the reverse process requires a substantial number of steps corresponding to the forward steps, which leads to slow sampling speed and expensive application cost of diffusion models [33].

C. Generative Adversarial Nets

The vanilla GANs proposed by Goodfellow et al. [23] consist of one discriminator (D) and one generator (G), which are trained simultaneously through a competitive process. The generator learn to capture the real data potential distribution $p_{data}(x)$ and generate new data instances with a random variable input z sampled from a prior distribution $p_z(z)$. While the discriminator aims to determine whether the input data is from the actual data or from the generator. The value function $V(G, D)$ can be defined as:

$$\begin{aligned} \min_G \max_D V(G, D) &= \mathbb{E}_{x \sim p_{data}(x)} [\log D(x)] \\ &+ \mathbb{E}_{z \sim p_z(z)} [\log(1 - D(G(z)))], \end{aligned} \quad (10)$$

where $G(\cdot)$ denotes a generator function, $D(\cdot)$ represent a discriminator function. The discriminator attempts to distinguish between real and synthetic data by maximizing the value function. Meanwhile, the generator is trained to produce high-quality fake data to fool the discriminator, thus to minimize the function.

Due to this unique min-max objective, GANs achieve exceptional sampling speed and generate high-quality samples. However, they often suffer from the problem of mode collapse, resulting in poor diversity in the generated data [34]. Therefore, there have been various improvements to enhance GANs performance.

Deep Convolutional GAN (DCGAN) [35] is a significant advancement among the framework of GANs to increase the stability of the training. It redesigns the architecture of G and D using Convolutional Neural Network (CNN) without changing the basic training strategy of the GANs. Wasserstein GAN (WGAN) [34] employs the Wasserstein distance as a replacement for the Jensen-Shannon divergence used in traditional GANs. This enhancement significantly mitigates the instability and mode collapse problems during the training of GANs.

Besides, the fundamental structure of GANs is sometimes insufficient in practice to fulfill task requirements. Conditional GAN (CGAN) [36] extend GANs to a conditional model by feeding extra conditional information y into the both G and D as additional input layer. The objective of CGAN can be defined as:

$$\min_G \max_D V(G, D) = \mathbb{E}_{x \sim p_{data}(x)} [\log D(x|y)] + \mathbb{E}_{z \sim p_z(z)} [\log(1 - D(G(z|y)))]. \quad (11)$$

ACGAN [37] is an extension of CGAN. In contrast to CGAN, the discriminator of ACGAN only takes data as input but produces an additional class label c for the input data. The extra classification objective L_c of the discriminator can be written as:

$$L_c = E(\log P(c|X_{real})) + E(\log P(c|X_{fake})), \quad (12)$$

where X_{real} and X_{fake} are both the input of discriminator.

D. Generative Model for Tabular Data

Modeling tabular data using generative models like VAE, GANs, and diffusion models is an active research area. Many approaches have been introduced to address the challenges of modeling tabular data, such as limited dataset size, mixed type of data, the absence of correlation among columns and rows, and so on. cWGAN [38] and SOS [27] provide solutions for addressing the long-standing problem of class imbalance in tabular data. cWGAN is primarily designed for binary classification and often experiences training instability. The SOS method requires multiple diffusion steps to generate minorities and necessitates training a separate model for each class. Tab-DDPM [39], CTGAN, and TVAE [40] effectively handle the complexities introduced by mixed-type characteristics in tabular data. STaSy [41] helps alleviate the challenges of learning the joint probability distribution of multiple columns in tabular data.

However, these models are not specifically tailored to address the imbalanced data problem.

E. The Combination of Diffusion Models and GANs

In recent years, there has been growing interest in combining GANs with diffusion models to leverage the strengths of both approaches. For instance, [42] proposes injecting noise into the discriminator with an adaptive noise schedule determined by a diffusion model to improve the stability of GANs during training. [33] argues that slow sampling is caused by the Gaussian assumption in the denoising step, which is only valid for small step sizes. To address this, they propose modeling each denoising step using a conditional GAN, allowing larger step sizes. Additionally, [43] introduces SynDiff, based on adversarial diffusion modeling, for improved performance in medical image translation inspired by [33]. These works primarily focus on generating image data under balanced datasets.

III. THE PROPOSED APPROACH

In this section, we present the motivation and provide an overview of the proposed method. Subsequently, we delve into the architectural details of the DGOT model, outlining its training objectives and procedures. Finally, we comprehensively explain the oversampling process that utilizes DGOT to tackle class imbalance issues in tabular data.

A. Motivation and Overall Workflow

In this paper, we employ a conditional diffusion model as the generator to mitigate mode collapse issues during the training of GANs as inspired by [33]. Additionally, we incorporate a Noise-Sensitive Auxiliary Classifier (NSAC) in the discriminator to enhance the conditional generation capability of the diffusive generator, which is particularly important for minority samples.

To ensure a clear and precise presentation of our approach, the notations used in this section are defined as follows. Consistent with the notation used in Section II, we use subscripts to denote the time step at which the data correspond to the diffusion process. This means we denoted the raw data as x_0 , the noisy version of x_0 as x_t , and the Gaussian distributed data as x_T . Then, we use superscripts to specifically distinguish their category (C) and source (R/F) based on the above definition.

For instance, the variable $x_t^{F,C}$ represents the noisy version of raw data at timestep t but generated from the generator under the control of C conditional information. Then, we can use $x_0^{F,C}$ to denote its corresponding raw data distribution produced by the generator. Thus, the fake (real) data of the class C can be defined as $x_0^{F,C}$ ($x_0^{R,C}$), while $x_t^{F,C}$ ($x_t^{R,C}$) represent noisy ones at time step t . In particular, as both $x_T^{R,C}$ and $x_T^{F,C}$ are equivalent to an isotropic Gaussian distribution, we simply denote it as $x_T \sim N(0, I)$. As illustrated in Fig. 1, our approach follows this workflow:

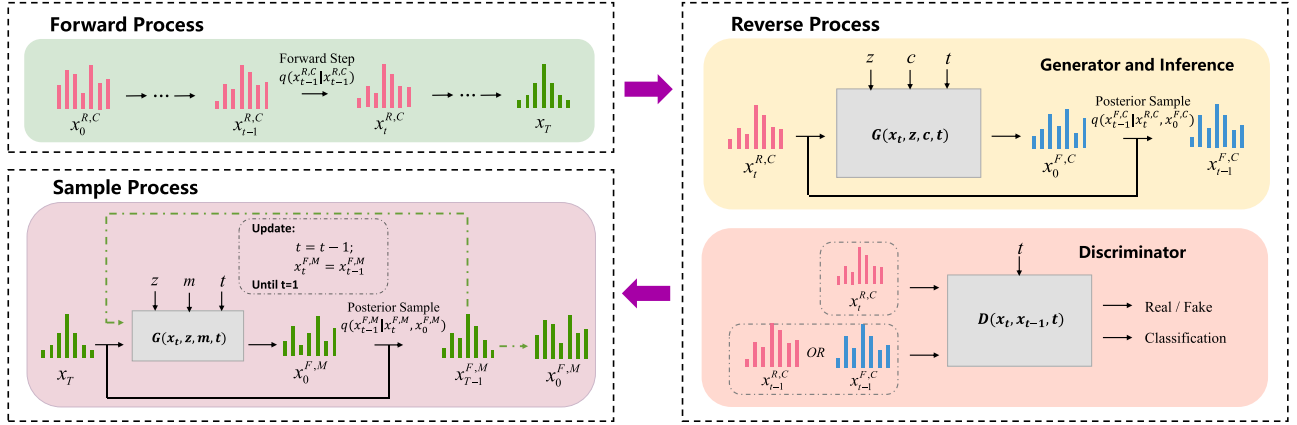


Fig. 1. The overall workflow of our method DGOT. It comprises the three fundamental components: Forward Process, Reverse Process and Sample Process.

- 1) In the first stage, the raw data distribution is transformed into a sequence of noisy data distributions under the control of a predefined variance schedule during the forward process.
- 2) Then, the model is trained by adversarial learning during the reverse process, in which the generator gradually learns to synthesize real data from a noisy one.
- 3) Finally, the trained generator gradually creates the minority class (M) data $x_0^{F,M}$ by taking a Gaussian noise as input after several reverse denoising steps.

B. Forward Process

We employ a forward diffusion process similar to regular diffusion models. This process is controlled by a predefined variance schedule $\beta(t) \in (0, 1)_{t=1}^T$, as indicated in (1) and (2). In our approach, the schedule will remain constant and is considered a hyperparameter during the training process. Then, the forward step for real data of class C can be rewritten as follows:

$$q(x_t^{R,C} | x_{t-1}^{R,C}) = N(x_t^{R,C}; \sqrt{1 - \beta(t)} x_{t-1}^{R,C}, \beta(t)I), \quad (13)$$

where $t \in [1, T]$.

Notably, in our approach, leveraging the superior capability of GANs to learn multimodal distributions, each diffusion step of our approach can be set with a significantly larger $\beta(t)$ than regular diffusion models. In other words, our model can achieve a higher sampling speed.

C. Generator and Inference Process

As described in (8), regular diffusion models are trained by learning a sequence of reverse denoised distributions $p(x_{t-1} | x_t)$. In this process, the model learns to progressively transform highly noisy data distribution $p(x_t)$ into slightly less noisy data distribution $p(x_{t-1})$ until reaching the pure data distribution $p(x_0)$.

In our approach, we construct a specialized generator to approximate the true denoising process. As mentioned, the model is tasked with conditionally generating minority class data until it reaches the desired number of instances. Therefore, we

incorporate class auxiliary information to guide the generation process. Then the generator $G(x_t, z, c, t; \theta_g)$ can be used to predict $x_{t-1}^{F,C}$ with the input of the noisy data x_t , the latent variate z , the class conditional information c and the specific timestep t :

$$x_{t-1}^{F,C} = G(x_t, z, c, t). \quad (14)$$

However, predicting x_{t-1} with different levels of data noise perturbation using a single network is challenging. The reason is that the model has to learn multiple distinct transformations between input and corresponding output distributions, which can be difficult to train.

Therefore, instead of directly predicting x_{t-1} at every noise level, we follow the standard diffusion modeling paradigm by allowing the generator predict the clean data x_0 . And then, the x_{t-1} will be sampled using the posterior distribution $q(x_{t-1} | x_t, x_0)$. Utilizing the Bayes' rule and Markov property of the forward process, the reverse distribution can be expressed as:

$$q(x_{t-1} | x_t, x_0) = \frac{q(x_t | x_{t-1})q(x_{t-1} | x_0)}{q(x_t | x_0)}, \quad (15)$$

Applying (2) and (3), we can express $q(x_{t-1} | x_t, x_0) = N(x_{t-1}; \tilde{\mu}_t(x_t, x_0), \tilde{\beta}(t)I)$ with the following definitions:

$$\begin{aligned} \tilde{\mu}_t &= \frac{\sqrt{\alpha_t}(1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t} x_t + \frac{\sqrt{\bar{\alpha}_{t-1}}\beta(t)}{1 - \bar{\alpha}_t} x_0, \\ \tilde{\beta}(t) &= \frac{1 - \bar{\alpha}_{t-1}}{1 - \bar{\alpha}_t} \beta(t). \end{aligned} \quad (16)$$

Therefore, in DGOT, we utilize a generator to predict the x_0 and infer the x_{t-1} using posterior distribution. Thus, the (14) can be rewritten as follows:

$$\begin{aligned} x_0^{F,C} &= G(x_t, z, c, t), \\ x_{t-1}^{F,C} &= q(x_{t-1} | x_t, x_0 = G(x_t, z, c, t)), \end{aligned} \quad (17)$$

The generator will be trained through an adversarial training process to capture the reverse distribution and generate data $x_{t-1}^{F,C}$ that matches the real data distribution.

D. Discriminator

We aim to train the generator using an adversarial loss, enabling it to match the reverse distributions and effectively generate minority class data to resolve the class imbalance issue. As previously discussed, incorporating an extra classifier structure can significantly improve the generator's conditional generation capability, particularly for the minority class. In DGOT, we will leverage this structure with the changes of features in the different noise levels of data to further strengthen the conditional generation capacity of the generator.

Specifically, we devise a distinct and innovative time-dependent discriminator with a noise-sensitive auxiliary classifier. This discriminator takes either $(x_t^{R,C}, x_{t-1}^{F,C})$ or $(x_t^{R,C}, x_{t-1}^{R,C})$ as input, then determines whether x_{t-1} is a credible denoised version of x_t and also classifies the input data. The discriminator $D(x_t, x_{t-1}, t; \theta_d)$ can be denoted as follow:

$$D : \{x_t, x_{t-1}, t\} \rightarrow \{D_{adv}(x_t, x_{t-1}, t), D_{cls}(x_t, x_{t-1}, t)\}, \quad (18)$$

where the term D_{adv} represents a probability distribution over sources, while D_{cls} denotes a probability distribution over the class labels.

As defined in (18), the discriminator is tasked with classifying data at each diffusion step. However, classifying corrupted data with a high SNR may not provide meaningful information for model training [44]. Therefore, it is crucial to analyze the features of tabular data from different classes at various noise levels and understand what insights the model can extract from them. This analysis will help us determine the priorities of classifying data at different time steps.

Here, we introduce a novel measurement for quantifying the differences between tabular data from different classes or at various noise levels. We first employ a pre-trained classifier model denoted as M_ϕ to calculate the conditional probability $P_\phi(C|x_t)$ for the noisy data x_t^C from the C category of raw data. This calculated value is termed the Conditional Probability Score (CPS). Then, we measure the dissimilarity between different data by comparing their CPS values, which we refer to as the Conditional Probability Distance (CPD).

Our investigation focuses on two specific cases: the distance D_1^t between raw data x_0^C and noisy data x_t^C within the same class C , as well as the distance D_2^t between noisy data from different classes a and b , namely x_t^a and x_t^b , particularly when they share the same noise level. These distances can be defined as follows:

$$\begin{aligned} D_1^t &= P_\phi(C|x_0^C) - P_\phi(C|x_t^C) \\ D_2^t &= 2P_\phi(C|x_t^C) - 1, \end{aligned} \quad (19)$$

where $t \in [1, 2, \dots, T]$ and $C = [a, b]$.

Specifically, to explore the features present in data at different noise levels, we quantified the CPD in the two cases using the Default dataset [45]. This dataset is divided into training and testing subsets. We first train the classifier model M_ϕ based on RandomForest algorithm [46] with the training data. Then, we use this pre-trained model to compute the average distances with the testing data. These average distances are graphically depicted

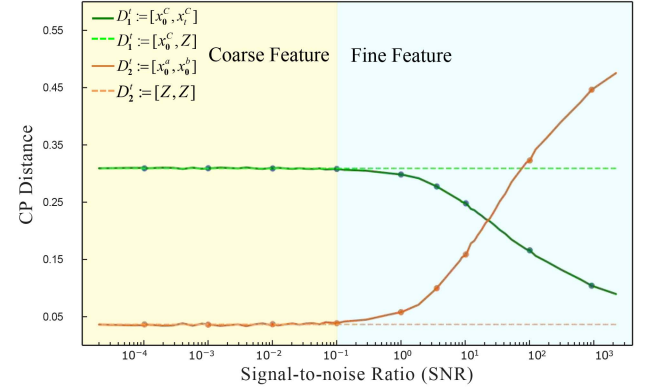


Fig. 2. The Conditional Probability Distance (CPD) for corrupted data at different Signal-to-Noise Ratios (SNRs). The solid green and orange lines varying with SNR represent the distances D_1^t and D_2^t , respectively. In contrast, the dotted green and orange lines illustrate the baseline distances calculated using random Gaussian data Z .

as functions of SNR in Fig. 2. Furthermore, for comparison, we assessed the distances using data samples drawn from a Gaussian distribution, denoted as Z .

As the SNR decreases, the noisy data progressively approaches a Gaussian distribution while far away from the raw data distribution simultaneously. As represented by the green solid line in Fig. 2, the distance D_1^t between x_t^C and x_0^C increases continuously and eventually stabilizes at a constant value, which is the same as the distance between pure Gaussian data Z and x_t^C . Similarly, the differences D_2^t between data from different classes, such as x_t^a and x_t^b , gradually diminish and converge to zero, as depicted by the orange line in Fig. 2.

These observations suggest that during the early stages of the reverse process, the noisy data retains coarse features that are challenging to distinguish. It poses a difficulty for the model in effectively recovering the data from its noisy version while preserving the correct class. However, as the process advances, the noisy data becomes more distinguishable. It indicates that similar specific and finer features appear in the noisy data as in the clean data. In other words, the reconstructed data gradually becomes identical with the original data. These fine features within the noisy data provide sufficient and valuable prior knowledge and guidance for model training and data generation. Hence, we can categorize the features contained in noisy data into two groups: coarse features and fine features, with the differentiation being associated with the change in SNRs.

Based on this analysis, we introduce a novel noise-sensitive auxiliary classifier (NSAC) to guide the model effectively in learning features at different steps. In our approach, we emphasize prioritizing learning for severely corrupted noisy data in the early stages of the reverse process. In contrast, for the later stages of the process, data with higher SNRs already encompass sufficient prior information, such as class information. Therefore, drawing inspiration from the widely used weighting scheme of the regular diffusion model, we propose a noise-sensitive weighting scheme for the classification loss of the discriminator. This scheme assigns higher weights to the data with higher

SNRs, which can be defined as follows:

$$\lambda_t = \frac{k_1}{1 + \ln(k_2 + \text{SNR}(t))}, \quad (20)$$

where the hyperparameters k_1 and k_2 are used to control the upper and lower limits of λ_t . Additionally, the value of k_2 impacts the sharpness of the weighting scheme and prevents exploding weights for extremely small SNRs.

E. Training Procedure

1) *Objective Function*: As previously explained, our overall loss consists of two components: the adversarial loss, denoted as L_{adv} , and the classification loss, L_{cls} . The adversarial loss addresses the min-max problem necessary for the generator to learn the reverse process. Utilizing (17) and (18), this can be expressed as:

$$L_{adv} = E_{q(x_t^C)}[E_{q(x_{t-1}^C|x_t^C)}[\log D_{adv}(x_{t-1}^{R,C}, x_t^{R,C}, t)] + E_{p_\theta(x_{t-1}^C|x_t^C)}[1 - \log D_{adv}(x_{t-1}^{F,C}, x_t^{R,C}, t)]], \quad (21)$$

where $p_\theta(x_{t-1}^C|x_t^C) := \int p(z)q(x_{t-1}^C|x_t^C, x_0^C = G(x_t, z, c, t))dz$, and $p(z) := N(0, 1)$. The discriminator (D) is optimized by maximizing this function to learn how to distinguish whether x_{t-1} can be recognized as a plausible denoised version of x_t . Meanwhile, the generator (G) endeavors to generate an indistinguishable x_0 to synthesize credible x_{t-1} by using (16) and thus minimizing the function.

Regarding L_{cls} , it can be decomposed into two classification loss components, one optimized for D and the other for G . The classification loss on real data is used for optimizing D . And the fake data is employed to optimize G . In detail, the former is defined as:

$$L_{cls}^R = E_{q(x_t^C)q(x_{t-1}^C|x_t^C)}[-\log D_{cls}(C|x_{t-1}^{R,C}, x_t^{R,C}, t)], \quad (22)$$

And the loss function of fake data can be represented as:

$$L_{cls}^F = E_{q(x_t^C)p_\theta(x_{t-1}^C|x_t^C)}[-\log D_{cls}(C|x_{t-1}^{F,C}, x_t^{R,C}, t)], \quad (23)$$

By minimize L_{cls}^R , D learns to classify a real data into the right category, while G tries to minimize the L_{cls}^F by generating data that can be classified correctly as the target class.

Therefore, with the definition of noise-sensitive weighting scheme λ_t , the complete objective functions for optimizing G and D can be expressed as follows, respectively:

$$L_D = \lambda_t L_{cls}^R - L_{adv}, \quad (24)$$

$$L_G = \lambda_t L_{cls}^F + L_{adv}, \quad (25)$$

Then, the DGOT model can be trained by minimizing these two objective functions.

2) *Network Structure*: The design of our generator is primarily inspired by the successful U-net frameworks utilized in DDPM, which consists of ResNet blocks and Attention blocks. However, in the case of DGOT, we've tailored the model suitable for our task by reducing its complexity to include only multiple residual blocks based on 1D-convolution. The transformer sinusoidal positional embedding technique embeds the integer diffusion timestep information. The prior input noise z and

Algorithm 1: Oversampling Procedure of DGOT.

Input: The imbalanced dataset S_0 , the number of each class $\{N_c\}_{c=1}^C$, the number of majority class N_{maj} , the dilatation coefficient d ;

```

1: for  $c=1$  to  $C$  do
2:   repeat
3:     Sample initial data  $X_T \sim N(0, I)$ ;
4:     for  $t=T$  to 1 do
5:       Sample latent variable  $z \sim N(0, I)$ ;
6:        $x_0 \leftarrow G(x_t, z, c, t)$ ;
7:        $x_{t-1} \leftarrow q(x_{t-1}|x_t, x_0)$ ;
8:     end for
9:      $S_0 \leftarrow [S_0, x_0]$ ;
10:     $N_c \leftarrow N_c + 1$ ;
11:  until  $N_c \geq d * N_{maj}$ 
12: end for
13:  $S_1 \leftarrow S_0$ ;

```

Output: Balancing Dataset S_1 ;

class information C are combined in joint hidden representation. Then, our generator integrates these three pieces of information through an adaptive group normalization (AdaGN) [47] layer after feature mapping.

Meanwhile, our discriminator design is similar to the generator's, as it's also based on ResNet blocks. However, instead of incorporating the timestep t through an AdaGN layer as in the generator, it is added in each ResNet block after being processed by mapping through a fully-connected network. Furthermore, in the discriminator, we replace group normalization layers and Swish activation functions used in the generator with batch normalization layers and Leaky ReLU activation functions to enhance training stability and model performance.

F. Oversampling Procedure

Algorithm 1 shows the specific oversampling procedure with the DGOT for imbalanced data. For an imbalanced dataset S_0 containing C classes, let N_{maj} represent the number of samples in the majority class. We perform oversampling on the imbalanced dataset by generating data with the trained generator until the number of samples in each class reaches $d * N_{maj}$, where d is a hyperparameter to control the maximum number of instances per class.

IV. EXPERIMENTS

A. Dataset Selection

In our experiments, we employed 22 real-world tabular datasets to evaluate the performance of our model. These datasets comprise 15 binary classification datasets and seven multi-classification datasets. All the datasets used in our experiments are publicly available, sourced from repositories including UCI [45] and the Python package imbalanced-learn [48]. Essential details for these datasets, including the selected target class (# T) as of minority class for binary classification, data size (#

TABLE I
DESCRIPTION OF THE DATASETS

Source	Data Name	# T	# S	# F	# C	IR
[45]	satimage	minority	6430	36	6	2.5
	students_dropout	minority	4424	36	3	2.8
	default	minority	30000	23	2	3.5
	Dry_Beans	minority	13611	16	7	6.8
	bank	minority	4521	16	2	7.7
	nursery	minority	12958	8	4	13.2
	abalone_15	≥ 15	4177	8	2	15.0
	abalone_6	≤ 6	4177	8	2	17.7
	polish_bankruptcy	minority	19967	64	2	45.2
	avila	minority	20850	10	11	97.4
	page-blocks	minority	5473	10	5	175.5
	firewall	minority	65532	11	4	697.0
[48]	optical_digits	8	5620	64	2	9.1
	satimage_4	4	6435	36	2	9.3
	pen_digits	5	10992	16	2	9.7
	sick_euthyroid	minority	3163	42	2	9.8
	isolet	A,B	7797	617	2	12.0
	thyroid_sick	minority	3772	52	2	15.0
	coil_2000	minority	9822	85	2	16.0
	wine_quality	≤ 4	4898	11	2	26.0
	letter_img	Z	20000	16	2	26.0
	abalone_19	19	4177	8	2	129.5

S), feature length (# F), the number of classes (# C), and the class imbalance ratio (IR), are summarized in Table I.

The class imbalance ratio (IR) is a commonly used metric to measure the degree of class imbalance in datasets and is defined as follows:

$$IR = \frac{N_{maj}}{N_{min}}, \quad (26)$$

where N_{maj} and N_{min} represent the number of majority and minority class instances, respectively. For multi-class datasets, IR is the ratio between the number of instances in the class with the most and those with the fewest instances.

B. Evaluation Metric

Our primary objective is to optimize the performance of minority classes while concurrently maintaining a commendable classification accuracy overall. The classification accuracy is expressed as:

$$Accuracy = \sum_{i=0}^m \frac{TP_i}{N}, \quad (27)$$

where TP_i signifies the count of accurately classified samples in the i -th class, m denotes the number of classes in the dataset, and N represents the total number of instances in the dataset. Although widely employed, classification accuracy alone may lack comprehensiveness in evaluating model performance, especially in imbalanced datasets. However, accuracy can indicate the influence of oversampling algorithms on the classification accuracy of majority class samples when used with other metrics. The highest accuracy result is achieved when the oversampling approach improves the recognition rate of the minority class while simultaneously maintaining the recognition rate of the majority class [49].

In addition to accuracy, we employ the F1-score and Matthews Correlation Coefficient (MCC) metrics to provide a more nuanced assessment of the model's predictive performance for each class. These metrics are calculated using correctly classified positive or negative class examples and the number of negative class examples misclassified as positive and vice versa. They can be defined as follows:

$$precision = \frac{TP}{TP + FP}$$

$$recall = \frac{TP}{TP + TN}$$

$$F1 = \frac{2 \times recall \times precision}{recall + precision}$$

$$MCC = \frac{TP \times TN - FP \times FN}{\sqrt{(TP+FN)(TP+FP)(TN+FP)(TN+FN)}}, \quad (28)$$

where TP, TN, FN, and FP represent the number of true positives, true negatives, false negatives, and false positives, respectively.

These metrics are particularly adapted to imbalanced data scenarios [30]. The F1-score, as the harmonic mean of precision and recall, captures a balanced measure of a model's effectiveness. Similarly, the MCC considers both true and false positives and negatives, presenting a balanced metric suitable for severely imbalanced data classification [50]. For the multi-class classification problem, we calculate metrics for each label and find their unweighted mean.

Therefore, we evaluate the performance of different resampling approaches by measuring the F1-score, MCC, and accuracy after using the data sampling methods to even the imbalanced tabular data distribution. For each of the metrics, higher values indicate superior model performance.

C. Comparison Algorithms

We assess the proposed DGOT approach by comparing it with various sampling algorithms, encompassing classical statistical and DGM-based methods. The selected classical sampling algorithms include SMOTE [15], BSMOTE [16], and SMOTETL [31], with the first two being oversampling techniques and the third serving as a hybrid-sampling approach. SMOTE is a well-established oversampling method that successfully resolved imbalanced dataset challenges across diverse research domains. BSMOTE and SMOTETL are two SMOTE variants that demonstrate superior performance in various scenarios [21]. The imbalanced-learn¹ package is utilized to implement these benchmark methods.

Additionally, we incorporate three deep generative models, including GANs, VAE, and diffusion models. CTGAN² [40] is a conditional GAN specifically designed for tabular data with mixed variable types. TVAE² [40] stands out as the state-of-the-art VAE-based model for generating tabular data. To our knowledge, no alternative VAE-like models outperform TVAE and have public source code. SOS³ [27] is a recent score-based

¹<https://imbalanced-learn.org/>

²<https://github.com/sdv-dev/SDGym>

³<https://github.com/JayoungKim408/SOS>

TABLE II
DEFINITIONS OF BASELINES

Algorithm Type	Name	Parameters
Classical Oversampling	SMOTE [15]	$k = 5$
	BSMOTE [16]	
Classical Hybridsampling	SMOTETL [31]	
Generative Oversampling	CTGAN [40]	<i>default</i>
	TVAE [40]	
	SOS [27]	$SDE = \text{Sub_VP}$

generative oversampling method tailored to address a class imbalance in tabular data, showcasing commendable performance relative to other methods. The implementations of these models are carried out using their official publicly available source codes. Table II summarizes the baseline methods, including their names, types, and hyperparameters.

Furthermore, we establish a control group to assess the impact of different sampling methods comprehensively. This group represents the original imbalanced tabular data without any intervention, serving as a baseline to showcase improvements achieved by applying sampling algorithms.

D. Experiment Design

In our experimental setup, we evaluate the performance of these resampling algorithms through several widely used machine learning classifiers. For each dataset, we employ a stratified 5-fold cross-validation approach, randomly dividing the dataset into five folds. During each iteration, one fold serves as the test set, and the remaining four folds constitute the training set. This process is repeated five times with a different fold as the test set. Resampling methods are then applied to balance the imbalance ratio within the training set. Subsequently, classification models are trained on the transformed training set and evaluated on the test data. We repeat this procedure with ten random seeds for each resampling method and report their average performance.

We choose XGBoost [51], K-Nearest-Neighbors (KNN) [52], Decision Tree Classifier (DTC) [53], Logistic Regression (Logit) [54] and Random Forest Classifier (RFC) [46] as the base classifiers. This selection encompasses a diverse set of classifiers, including ensemble, distance-based, tree-based, and regression-based models, providing a multifaceted evaluation of the resampling algorithms' performance [38].

For fair comparisons, handling hyperparameters for classifiers and resampling methods is crucial. Due to computational constraints, we utilize a set of reasonable hyperparameters for each baseline and classifier according to their papers or codes, as detailed in Tables II and III. For example, SOS is conducted based on the Stochastic Differential Equation (SDE) types of sub-VP, yielding excellent results in their experiments.

Considering the novelty and complexity of our method, an exhaustive tuning of all hyperparameters for each dataset might give it an advantage over benchmarks. To ensure fairness in the

TABLE III
PARAMETERS OF MACHINE LEARNING CLASSIFIERS

Classifier	Parameters
XGBoost [51]	$n_estimators = 100$
KNN [52]	$k = 5$
DTC [53]	$criterion = gini$
Logit [54]	$max_iter = 500, penalty = l2$
RF [46]	$n_estimators = 100$

comparison experiments, we adopt the same set of hyperparameters across all experiments for our method. Detailed information on implementing the noise schedule and other specifics of our model training are presented in Section B of the supplementary. It is acknowledged that fine-tuning DGOT for specific datasets could enhance its performance. We will explore this aspect in parameter sensitivity experiments, showcasing the best possible performance through fine-tuning. All the experiments were implemented using Python software and executed on an i9-13900 k CPU and NVIDIA GeForce 3090. Our code has been made publicly available at <https://github.com/MLneuers/DGOT>.

E. Results and Analyses

Using three performance metrics, we compare the six oversampling methods in conjunction with five different classifiers on 22 datasets. Tables IV–VI reports the accuracy, F1-score, and MCC comparison results. As mentioned earlier, 'IDENTITY' means we do not over-sample minor classes, train classifiers, and evaluate with testing data. Therefore, the 'IDENTITY' score can be considered a minimum requirement upon oversampling. For each table cell, the average mean and standard deviation of the evaluation metric are first recorded, the rank among all methods is followed in the bracket, and the best rank is highlighted in bold. The last row shows each method's average rank over all datasets.

Classical-based algorithms, including SMOTE, BSMOTE, and SMOTETL, demonstrate their effectiveness in improving the ability of minority classes to some extent. For instance, SMOTE achieved the best F1-score once and the best MCC twice, while BSMOTE attained the best on both F1-score and MCC once. Generally, SMOTE and SMOTETL outperformed IDENTITY in average rank across all datasets regarding F1-score and MCC. However, they all exhibit severe degradation compared to IDENTITY in accuracy, indicating an adverse impact on the classification accuracy of majority class samples.

CTGAN and TVAE are two deep generative models designed for tabular data synthesis but have yet to be specifically tailored to address the imbalanced data problem. Consequently, they are observed to be more effective on datasets with lower IR than those with higher ones. CTGAN performs worse than TVAE and other algorithms, possibly attributable to the mode collapse problem associated with GANs. Among the baselines, SOS consistently demonstrates the best results in many cases, and

TABLE IV
AVERAGE PERFORMANCE ON 22 DATASETS FOR ACCURACY

Dataset	IDENTITY	SMOTE	BSMOTE	SMOTETL	CTGAN	TVAE	SOS	DGOT
satimage	0.8876±0.0350(2)	0.8852±0.0305(4)	0.8769±0.0340(8)	0.8852±0.0302(5)	0.8832±0.0323(6)	0.8811±0.0335(7)	0.8892±0.0302(1)	0.8868±0.0353(3)
students_dropout	0.7300±0.0412(3)	0.7111±0.0549(7)	0.7122±0.0565(6)	0.7062±0.0558(8)	0.7226±0.0435(4)	0.7214±0.0475(5)	0.7338±0.0402(1)	0.7320±0.0376(2)
default	0.7926±0.0339(1)	0.7606±0.0463(6)	0.7583±0.0486(7)	0.7608±0.0455(5)	0.7562±0.0512(8)	0.7882±0.0339(3)	0.7700±0.0353(4)	0.7922±0.0325(2)
Dry_Beans	0.9195±0.0102(2)	0.9189±0.0122(4)	0.9188±0.0105(5)	0.9193±0.0093(3)	0.9180±0.0129(6)	0.9161±0.0118(8)	0.9172±0.0123(7)	0.9220±0.0086(1)
bank_small	0.8884±0.0147(1)	0.8534±0.0349(6)	0.8555±0.0342(4)	0.8535±0.0356(5)	0.8480±0.0422(7)	0.8588±0.0348(3)	0.8400±0.0406(8)	0.8867±0.0145(2)
optical_digits	0.9787±0.0138(2)	0.9786±0.0172(3)	0.9730±0.0225(6)	0.9785±0.0171(4)	0.9567±0.0514(8)	0.9725±0.0181(7)	0.9783±0.0197(5)	0.9811±0.0123(1)
satimage_4	0.9284±0.0195(1)	0.8514±0.1104(7)	0.8513±0.1068(8)	0.8518±0.1107(6)	0.9159±0.0295(3)	0.8641±0.1077(5)	0.8772±0.0687(4)	0.9261±0.0269(2)
pen_digits	0.9889±0.0139(2)	0.9837±0.0248(3)	0.9681±0.0542(7)	0.9836±0.0249(5)	0.9677±0.0531(8)	0.9700±0.0490(6)	0.9837±0.0244(4)	0.9892±0.0142(1)
sick_euthyroid	0.9554±0.0233(2)	0.9395±0.0405(4)	0.9436±0.0361(3)	0.9392±0.0405(5)	0.9169±0.0619(7)	0.9162±0.0447(8)	0.9275±0.0408(6)	0.9604±0.0198(1)
isolet	0.9683±0.0167(3)	0.9563±0.0306(6)	0.9530±0.0293(8)	0.9562±0.0306(7)	0.9668±0.0154(4)	0.9638±0.0188(5)	0.9692±0.0179(2)	0.9694±0.0143(1)
nursery	0.8536±0.2205(2)	0.8281±0.2591(4)	0.8272±0.2603(5)	0.8270±0.2589(6)	0.7689±0.2212(8)	0.8014±0.2322(7)	0.8565±0.2157(1)	0.8447±0.2401(3)
thyroid_sick	0.9718±0.0154(2)	0.9510±0.0433(4)	0.9528±0.0427(3)	0.9508±0.0441(5)	0.9214±0.0636(8)	0.9444±0.0266(6)	0.9373±0.0393(7)	0.9751±0.0132(1)
abalone_15	0.9303±0.0128(2)	0.8779±0.0332(6)	0.8897±0.0267(4)	0.8774±0.0332(7)	0.8863±0.0488(5)	0.9081±0.0270(3)	0.8710±0.0307(8)	0.9314±0.0128(1)
coil_2000	0.9275±0.0163(1)	0.8211±0.0960(8)	0.8386±0.0786(6)	0.8212±0.0958(7)	0.9139±0.0280(5)	0.9274±0.0166(2)	0.9153±0.0179(4)	0.9269±0.0161(3)
abalone_6	0.9854±0.0032(2)	0.9787±0.0073(5)	0.9758±0.0118(7)	0.9785±0.0071(6)	0.9828±0.0070(3)	0.9803±0.0104(4)	0.9748±0.0074(8)	0.9868±0.0036(1)
wine_quality	0.9574±0.0134(2)	0.8697±0.0779(6)	0.8843±0.0762(4)	0.8690±0.0790(7)	0.8051±0.1361(8)	0.9481±0.0194(3)	0.8703±0.0553(5)	0.9622±0.0076(1)
letter_img	0.9943±0.0060(2)	0.9861±0.0203(4)	0.9820±0.0283(6)	0.9862±0.0203(3)	0.9663±0.0572(8)	0.9796±0.0299(7)	0.9829±0.0195(5)	0.9947±0.0053(1)
bankruptcy	0.9769±0.0061(2)	0.8911±0.0978(7)	0.9130±0.0774(6)	0.8905±0.0973(8)	0.9515±0.0453(5)	0.9768±0.0063(3)	0.9758±0.0064(4)	0.9774±0.0050(1)
avila	0.8651±0.1876(2)	0.8532±0.2177(4)	0.8269±0.2611(6)	0.8527±0.2177(5)	0.8193±0.2535(8)	0.8239±0.2334(7)	0.8562±0.1981(3)	0.8678±0.1830(1)
abalone_19	0.9902±0.0047(2)	0.9220±0.0684(7)	0.9525±0.0549(4)	0.9219±0.0682(8)	0.9280±0.0637(6)	0.9903±0.0047(1)	0.9480±0.0427(5)	0.9883±0.0067(3)
page-blocks	0.9637±0.0092(2)	0.9363±0.0388(5)	0.9427±0.0328(4)	0.9354±0.0389(6)	0.9063±0.0767(8)	0.9237±0.0402(7)	0.9573±0.0189(3)	0.9691±0.0088(1)
firewall	0.9924±0.0095(2)	0.9679±0.0406(5)	0.9417±0.0990(8)	0.9679±0.0406(6)	0.9665±0.0525(7)	0.9685±0.0504(4)	0.9926±0.0094(1)	0.9900±0.0145(3)
Average Rank	1.91	5.23	5.68	5.77	6.36	5.05	4.36	1.63

TABLE V
AVERAGE PERFORMANCE ON 22 DATASETS FOR F1-SCORE

Dataset	IDENTITY	SMOTE	BSMOTE	SMOTETL	CTGAN	TVAE	SOS	BTGD
satimage	0.8626±0.0461(5)	0.8693±0.0333(1)	0.8593±0.0379(7)	0.8692±0.0331(2)	0.8591±0.0413(8)	0.8619±0.0375(6)	0.8643±0.0400(4)	0.8660±0.0420(3)
students_dropout	0.6451±0.0487(7)	0.6552±0.0517(4)	0.6561±0.0540(3)	0.6525±0.0537(5)	0.6518±0.0513(6)	0.6581±0.0443(2)	0.6415±0.0486(8)	0.6632±0.0408(1)
default	0.6485±0.0283(7)	0.6530±0.0290(5)	0.6514±0.0301(6)	0.6532±0.0285(4)	0.6483±0.0300(8)	0.6688±0.0298(2)	0.6579±0.0286(3)	0.6690±0.0274(1)
Dry_Beans	0.9309±0.0087(2)	0.9304±0.0108(5)	0.9306±0.0093(4)	0.9307±0.0086(3)	0.9293±0.0118(7)	0.9269±0.0111(8)	0.9301±0.0110(6)	0.9327±0.0080(1)
bank_small	0.6620±0.0443(7)	0.6764±0.0337(5)	0.6771±0.0365(4)	0.6822±0.0343(3)	0.6619±0.0412(8)	0.6658±0.0394(6)	0.6873±0.0388(2)	0.6950±0.0332(1)
optical_digits	0.9375±0.0401(5)	0.9429±0.0425(3)	0.9281±0.0539(6)	0.9424±0.0426(4)	0.8980±0.1012(8)	0.9222±0.0487(7)	0.9442±0.0477(2)	0.9448±0.0352(1)
satimage_4	0.7366±0.1350(3)	0.7297±0.1101(6)	0.7278±0.1081(8)	0.7301±0.1104(5)	0.7296±0.1329(7)	0.7332±0.1143(4)	0.7422±0.0975(2)	0.7625±0.1018(1)
pen_digits	0.9663±0.0434(2)	0.9601±0.0576(3)	0.9328±0.1050(8)	0.9598±0.0578(5)	0.9336±0.1002(7)	0.9380±0.0923(6)	0.9600±0.0569(4)	0.9689±0.0408(1)
sick_euthyroid	0.8385±0.1054(5)	0.8536±0.0801(3)	0.8582±0.0764(2)	0.8531±0.0797(4)	0.7857±0.1340(8)	0.8022±0.0848(7)	0.8324±0.0731(6)	0.8754±0.0657(1)
isolet	0.8819±0.0589(3)	0.8739±0.0659(5)	0.8581±0.0676(8)	0.8734±0.0660(6)	0.8754±0.0552(4)	0.8731±0.0593(7)	0.8920±0.0578(1)	0.8844±0.0539(2)
nursery	0.8047±0.2510(3)	0.7853±0.2590(4)	0.7836±0.2599(6)	0.7845±0.2597(5)	0.6754±0.1949(8)	0.7168±0.2141(7)	0.8095±0.2487(1)	0.8059±0.2523(2)
thyroid_sick	0.8440±0.1062(5)	0.8447±0.1044(4)	0.8473±0.1039(2)	0.8458±0.1057(3)	0.7587±0.1270(8)	0.8096±0.0741(7)	0.8110±0.0856(6)	0.8813±0.0692(1)
abalone_15	0.6048±0.0320(8)	0.6422±0.0248(4)	0.6547±0.0262(3)	0.6421±0.0271(5)	0.6118±0.0410(7)	0.6231±0.0362(6)	0.6575±0.0373(2)	0.6624±0.0473(1)
coil_2000	0.5155±0.0199(7)	0.5254±0.0255(5)	0.5339±0.0191(1)	0.5261±0.0256(3)	0.5198±0.0176(6)	0.5151±0.0194(8)	0.5335±0.0167(2)	0.5254±0.0207(4)
abalone_6	0.9264±0.0169(2)	0.9078±0.0261(5)	0.8996±0.0371(7)	0.9072±0.0253(6)	0.9209±0.0251(3)	0.9108±0.0349(4)	0.8958±0.0240(8)	0.9341±0.0184(1)
wine_quality	0.5760±0.0645(7)	0.5817±0.0550(6)	0.5929±0.0536(4)	0.5818±0.0541(5)	0.5366±0.0786(8)	0.5995±0.0664(2)	0.5957±0.0547(3)	0.6524±0.0497(1)
letter_img	0.9554±0.0504(2)	0.9326±0.0830(4)	0.9214±0.1019(5)	0.9329±0.0829(3)	0.8959±0.1375(8)	0.9091±0.1033(7)	0.9146±0.0777(6)	0.9618±0.0389(1)
bankruptcy	0.5874±0.0939(5)	0.6104±0.1099(1)	0.6076±0.0945(3)	0.6080±0.1083(2)	0.5780±0.0711(8)	0.5870±0.0938(6)	0.5852±0.0918(7)	0.5939±0.0958(4)
avila	0.8046±0.2885(6)	0.8472±0.2331(1)	0.8192±0.2759(4)	0.8466±0.2331(2)	0.7862±0.2745(8)	0.7890±0.2580(7)	0.8058±0.2820(5)	0.8393±0.2249(3)
abalone_19	0.5083±0.0272(5)	0.5122±0.0473(2)	0.5084±0.0361(4)	0.5105±0.0460(3)	0.4929±0.0270(8)	0.5083±0.0270(6)	0.5062±0.0129(7)	0.5277±0.0463(1)
page-blocks	0.8020±0.0457(2)	0.7673±0.0899(4)	0.7846±0.0786(3)	0.7669±0.0888(5)	0.6839±0.1274(8)	0.6928±0.0921(7)	0.7593±0.1248(6)	0.8405±0.0210(1)
firewall	0.8257±0.0817(2)	0.7659±0.0488(7)	0.7658±0.1303(8)	0.7670±0.0498(6)	0.7883±0.0872(5)	0.7913±0.0855(4)	0.8225±0.0785(3)	0.8468±0.0940(1)
Average Rank	4.55	3.95	4.82	4.04	7.09	5.72	4.27	1.54

its synthesis quality appears more stable overall. However, it also fails in some cases especially on heavily imbalanced data.

Our method outperforms every other algorithm in all three metrics compared to these baselines. The DGOT achieves the best performance on 14 out of 22 datasets concerning accuracy, 18 out of 22 for F1-score, and 12 out of 22 for MCC. More importantly, in comparison with ‘IDENTITY,’ our method demonstrates improvements in almost all indicators across 22 datasets, except for five instances regarding accuracy. In contrast, other oversampling algorithms occasionally fail, and in some cases, they may significantly impact the accuracy of the classifier.

Compared with ‘IDENTITY,’ all methods failed on the page_blocks dataset except for our DGOT. This phenomenon may be because the difficulty of synthesizing data based on the original data increases with the aggravation of the imbalance in the original data distribution and the number of

sample categories in the dataset. Especially when a small portion of classes have many sample points while others have very few; this is termed a long-tailed class distribution [55]. In our experiments, for page_blocks dataset with five classes, the proportion of each category in the total sample is 0.898: 0.060: 0.021: 0.016: 0.005, which can almost be regarded as a long-tailed distribution. The unique performance on the severely imbalanced dataset such as page_blocks dataset verifies the confirmed efficiency and superiority of the proposed DGOT.

Unlike metrics such as F1-score and MCC, accuracy is more influenced by the majority class. The highest accuracy result is achieved when the oversampling approach improves the recognition rate of the minority class while simultaneously maintaining the recognition rate of the majority class. In other words, the best result is only achieved when newly generated minority samples are genuinely situated within the minority

TABLE VI
AVERAGE PERFORMANCE ON 22 DATASETS FOR MCC

Dataset	IDENTITY	SMOTE	BSMOTE	SMOTETL	CTGAN	TVAE	SOS	BTDG
satimage	0.8612±0.0432(2)	0.8594±0.0373(4)	0.8494±0.0415(8)	0.8593±0.0369(5)	0.8559±0.0399(6)	0.8539±0.0408(7)	0.8633±0.0373(1)	0.8602±0.0435(3)
students_dropout	0.5522±0.0676(3)	0.5344±0.0843(7)	0.5358±0.0867(6)	0.5286±0.0849(8)	0.5452±0.0724(5)	0.5465±0.0742(4)	0.5590±0.0664(1)	0.5580±0.0623(2)
default	0.3318±0.0620(3)	0.3216±0.0650(5)	0.3194±0.0658(7)	0.3214±0.0638(6)	0.3156±0.0688(8)	0.3497±0.0664(2)	0.3228±0.0607(4)	0.3529±0.0624(1)
Dry_Beans	0.9027±0.0123(2)	0.9020±0.0148(5)	0.9021±0.0127(4)	0.9025±0.0112(3)	0.9010±0.0155(6)	0.8987±0.0142(8)	0.9000±0.0148(7)	0.9057±0.0104(1)
bank_small	0.3551±0.0718(6)	0.3710±0.0727(5)	0.3711±0.0781(4)	0.3822±0.0707(3)	0.3381±0.0795(8)	0.3441±0.0810(7)	0.4001±0.0646(1)	0.3991±0.0652(2)
optical_digits	0.8774±0.0795(5)	0.8886±0.0820(3)	0.8606±0.1035(6)	0.8876±0.0823(4)	0.8061±0.1881(8)	0.8470±0.0970(7)	0.8916±0.0911(1)	0.8915±0.0698(2)
satimage_4	0.4920±0.2530(7)	0.5172±0.1555(3)	0.5115±0.1567(5)	0.5174±0.1561(2)	0.4647±0.2607(8)	0.5016±0.1759(6)	0.5116±0.1773(4)	0.5339±0.2001(1)
pen_digits	0.9335±0.0854(2)	0.9246±0.1064(4)	0.8766±0.1880(8)	0.9241±0.1068(5)	0.8806±0.1742(7)	0.8897±0.1576(6)	0.9246±0.1053(3)	0.9379±0.0815(1)
sick_euthyroid	0.6958±0.1882(5)	0.7233±0.1442(3)	0.7288±0.1410(2)	0.7226±0.1429(4)	0.5812±0.2626(8)	0.6217±0.1565(7)	0.6872±0.1260(6)	0.7541±0.1283(1)
isolet	0.7713±0.1146(3)	0.7629±0.1123(4)	0.7314±0.1220(8)	0.7620±0.1127(5)	0.7585±0.1070(6)	0.7517±0.1170(7)	0.7857±0.1151(1)	0.7765±0.1030(2)
nursery	0.7848±0.3241(2)	0.7649±0.3480(4)	0.7642±0.3488(5)	0.7636±0.3476(6)	0.6909±0.2960(8)	0.7294±0.3138(7)	0.7894±0.3166(1)	0.7844±0.3281(3)
thyroid_sick	0.7116±0.1870(4)	0.7096±0.1840(5)	0.7131±0.1846(2)	0.7122±0.1859(3)	0.5487±0.2364(8)	0.6358±0.1379(7)	0.6525±0.1445(6)	0.7681±0.1330(1)
abalone_15	0.2495±0.0961(7)	0.3229±0.0566(5)	0.3384±0.0591(2)	0.3229±0.0608(4)	0.2434±0.0686(8)	0.2734±0.0744(6)	0.3665±0.0675(1)	0.3380±0.0917(3)
coil_2000	0.0662±0.0377(6)	0.1109±0.0482(3)	0.1155±0.0454(1)	0.1121±0.0461(2)	0.0644±0.0351(8)	0.0654±0.0374(7)	0.0829±0.0320(5)	0.0873±0.0383(4)
abalone_6	0.8538±0.0337(2)	0.8238±0.0464(5)	0.8109±0.0622(7)	0.8227±0.0449(6)	0.8462±0.0451(3)	0.8270±0.0620(4)	0.8045±0.0409(8)	0.8693±0.0365(1)
wine_quality	0.2048±0.1327(7)	0.2262±0.0644(6)	0.2416±0.0647(3)	0.2271±0.0601(5)	0.1755±0.0717(8)	0.2292±0.1291(4)	0.2623±0.0796(2)	0.3445±0.0944(1)
letter_img	0.9135±0.0961(2)	0.8787±0.1406(4)	0.8595±0.1715(5)	0.8792±0.1404(3)	0.8245±0.2127(8)	0.8357±0.1750(7)	0.8457±0.1309(6)	0.9240±0.0776(1)
bankruptcy	0.2075±0.1944(5)	0.2997±0.1526(1)	0.2739±0.1456(3)	0.2947±0.1499(2)	0.1896±0.1435(8)	0.2067±0.1943(6)	0.1964±0.1917(7)	0.2231±0.1928(4)
avila	0.8164±0.2581(4)	0.8280±0.2515(1)	0.8013±0.2942(6)	0.8275±0.2513(2)	0.7874±0.2874(8)	0.7896±0.2728(7)	0.8062±0.2692(5)	0.8265±0.2400(3)
abalone_19	0.0220±0.0611(6)	0.0805±0.0904(1)	0.0484±0.0778(5)	0.0766±0.0898(2)	0.0195±0.0544(8)	0.0219±0.0609(7)	0.0591±0.0677(4)	0.0649±0.1057(3)
page-blocks	0.8014±0.0611(2)	0.7535±0.1046(5)	0.7695±0.0940(4)	0.7521±0.1030(6)	0.6828±0.1449(8)	0.7060±0.1055(7)	0.7777±0.0962(3)	0.8390±0.0465(1)
firewall	0.9871±0.0160(2)	0.9492±0.0619(5)	0.9093±0.1512(8)	0.9492±0.0619(6)	0.9477±0.0801(7)	0.9508±0.0764(4)	0.9875±0.0158(1)	0.9833±0.0238(3)
Average Rank	3.95	4.00	4.95	4.18	7.27	6.09	3.55	2

TABLE VII
MEAN RANKS AND STATISTICALLY TEST RESULT

	IDENTITY	SMOTE	BSMOTE	SMOTETL	CTGAN	TVAE	SOS	DGOT
Acc	1.91	5.23-	5.68-	5.77-	6.36-	5.05-	4.36-	1.63
F1	4.55-	3.95-	4.82-	4.04-	7.09-	5.72-	4.27-	1.54
MCC	3.95	4.00-	4.95-	4.18-	7.27-	6.09-	3.55	2
The critical value of Friedman Test: $F(7,147)=2.07$, ($\alpha=0.05$)								
Actual value of Accuracy: $F_F=23.65$ (reject);								
Actual value of F1-score: $F_F=15.16$ (reject);								
Actual value of MCC: $F_F=16.11$ (reject)								
the Bonferroni-Dunn test: $CD=1.99$, ($k=8, \alpha=0.05$)								

space [49]. It is particularly challenging in the presence of class overlap, small disjuncts, and noise examples. However, it's worth noting that only our method shows a clear improvement in accuracy compared with 'IDENTITY,' while others do not. Generally, the generative oversampling methods demonstrate better effectiveness than classical methods in solving this problem. Therefore, it indicates that only DGOT can improve the recognition rate of the minority class while maintaining the majority class's performance at the same time.

F. Statistical Analysis

1) *Friedman Test and Post-Hoc Bonferroni-Dunn Test*: We conducted the non-parametric statistical Friedman test and the post-hoc Bonferroni-Dunn test at a significance level of $\alpha = 0.05$ to evaluate the statistically significant differences in the results and compare the DGOT with other approaches. Compared with the critical values of the Friedman test, if the actual value is larger than it, we can reject the null hypothesis of equal performance between the compared algorithms among the metrics, marking this result with a *reject* in brackets. As seen in

TABLE VIII
WILCOXON RANK TEST ($\alpha = 0.05$)

Acc		MCC	
DGOT vs.	p-V	DGOT vs.	p-V
IDENTITY	0.18	SOS	0.0000003
		IDENTITY	0.00006

Table VII, there is a significant difference between the compared groups for all evaluation metrics.

Then, we used the post-hoc Bonferroni-Dunn test to evaluate the performance of our method. If the gap between two mean ranks is larger than the Critical Difference (CD), our method is believed to outperform the compared one, so we marked a symbol - (one minus sign) to the right of the compared result. As summarized in Table VII, our DGOT outperforms other methods on almost all metrics in the post-hoc test, except when compared with 'SOS' on MCC and 'IDENTITY' on accuracy and MCC, possibly due to the Bonferroni test is generally conservative [56].

2) *Wilcoxon Paired Signed-Rank Test*: As shown in Table VIII, we apply the Wilcoxon signed rank hypothesis test as a complement to test the significance difference between our method and the baselines not significant in the Bonferroni-Dunn test. If the p-value is below 0.05, the two methods have a significant difference. In other words, our method outperforms the compared one, and we highlight this p-value as bold. The result shows that our method outperforms the 'SOS' and 'IDENTITY' for MCC but does not outperform the 'IDENTITY' for accuracy. The reason is that, for some datasets, the average accuracy results between our method and 'IDENTITY' are very close.

G. Ablation Study

Our DGOT incorporates several elements different from other generative oversampling methods. In this study, we conduct a

TABLE IX
ABLATION STUDY RESULTS FOR F1-SCORE

	default	Dry_Beans	abalone_19	page-blocks
IDENTITY	0.6485±0.0283	0.9309±0.0087	0.5083±0.0272	0.8020±0.0457
Non-diff. module	0.6533±0.0291	0.9303±0.0105	0.5097±0.0325	0.8060±0.0491
w/o adv. loss	0.6139±0.0918	0.7842±0.2908	0.5041±0.0179	0.7021±0.2578
w/o NSAC	0.6609±0.0254	0.9305±0.0106	0.5138±0.0270	0.7791±0.1174
w/o NS	0.6677±0.0285	0.9319±0.0081	0.5116±0.0368	0.8368±0.0261
ours	0.6690±0.0274	0.9327±0.0080	0.5277±0.0463	0.8405±0.0210

set of ablation studies to evaluate these elements' importance in DGOT systematically. We investigate the impact of constructing the generator based on diffusion models by removing the diffusive module from our model. Specifically, we train a regular GAN with our network structure, similar to the ACGAN. Besides, to underscore the significance of the adversarial learning process, we implemented a classifier-free guidance conditional diffusion model inspired by the gradient of an implicit classifier, following the protocol in [57]. The classifier-free guidance can avoid the risks associated with a pre-trained classifier while maintaining sample quality. This approach is similar to ours, as we do not rely on an independently trained pre-trained classifier as a guide. As a supplement, Table S2 in Section-C of the supplementary material compares our approach with the classifier guidance diffusion model. The conditional diffusion model is conducted with the same network structure as our DGOT.

In addition, we compared our model against the DGOT without NSAC module to assess the effectiveness of the proposed NSAC module. In this 'w/o NSAC' experiment, we conduct it by removing the entire auxiliary classifier, which can also be regarded as setting λ_t to 0. Furthermore, we also conduct another group by removing the Noisy Sensitive (NS) weighting scheme and assigning weights equally to the L_{cls} computed at each time step, termed as 'w/o NS.' The sum of all weights for the L_{cls} used in both the 'w/o NS' and our methods are restricted to be the same to avoid the influence caused by the different proportion between L_{cls} and L_{adv} losses.

We conducted experiments on four representative datasets, comprising two binary classification datasets and two multi-classification datasets, covering both low and high IR cases. Table IX shows the result for the F1-score, and results for ACC and MCC are provided in Tables S3 and S4 of the supplemental material. These results show that our DGOT achieved significantly higher performance than all variants in each metric. The regular conditional diffusion model trained with 'T=4' failed on all datasets, leading to a substantial degradation in classifier performance. The diffusion and NSAC modules are crucial for DGOT, particularly proving more useful when dealing with extremely imbalanced datasets. The failure of DGOT without the diffusion module on the severely imbalanced datasets verifies the promotion of stability and sample quality introduced by the diffusion module. Meanwhile, the results of 'w/o NS' demonstrate the improvement in sample quality due to the noisy weighting scheme. We will further discuss the impact of NS in Section IV-H.

TABLE X
SENSITIVITY ON HYPERPARAMETERS

Hyperparameter Settings		Acc	F1	MCC
T	2	0.9302±0.0130	0.5263±0.02881	0.0988±0.0494
	4	0.9269±0.0161	0.5254±0.0207	0.0873±0.0383
	10	0.9289±0.0143	0.5258±0.0301	0.0840±0.0513
(k_1, k_2)	(1,2)	0.9293±0.0130	0.5283±0.0270	0.0901±0.0460
	(1,1)	0.9269±0.0161	0.5254±0.0207	0.0873±0.0383
	(2,1)	0.9291±0.0151	0.5273±0.0216	0.0961±0.0505
(β_1, β_2)	(15,0.1)	0.9301±0.0127	0.5308±0.0275	0.0999±0.0411
	(20,0.1)	0.9269±0.0161	0.5254±0.0207	0.0873±0.0383
	(10,0.1)	0.9297±0.0128	0.5291±0.0290	0.0947±0.0503
Channel multiplier of the Generator	(1,2)	0.9264±0.0149	0.5291±0.0186	0.0889±0.0373
	(1,2,2)	0.9269±0.0161	0.5254±0.0207	0.0873±0.0383
	(1,2,4)	0.9268±0.0154	0.5252±0.0204	0.0829±0.0351

H. Sensitivity on Hyperparameters

Our approach uses the same set of parameters for each study dataset to ensure the fairness of comparison experiments. However, some datasets can get better results with other parameters. This section summarizes the sensitivity results concerning some key hyperparameters on the 'coil_2000' dataset, which our method performs poorly.

As Table X shown, T means the number of time steps, k_1 and k_2 are the hyperparameters of schedule λ_t , β_1 and β_2 affect the order and size of the noise schedule. Generally, the model produces better results in 'coil_2000' by fine-tuning the main hyperparameters. The best settings for these parameters vary from one dataset to another, but generally, it is recommended that the number of T might increase with the degree of data imbalance. The hyperparameters (k_1, k_2) not only balance the combination of L_{cls} and L_{adv} , but are also related to the variation tendency of the NSAC. The parameters (β_1, β_2) influence the arrangements of the variance schedule, and the different channel multipliers of the generator cause various complexities in the model architecture, which should be matched with the scale of the dataset.

V. CONCLUSION

This paper has proposed a new generative oversampling framework called DGOT, combining GANs and diffusion models to balance the imbalanced tabular data. DGOT contains a diffusion model-based generator and a time-dependent discriminator with a NSAC. The discriminator and NSAC help the generator learn a conditional reverse diffusion process by adversarial learning. Then, the trained generator is used to over-sample the minority class to even the imbalanced data distribution.

Our proposed model enjoys better sample diversity, training stability, and conditional generation capability than traditional GANs and diffusion models. With the comparison between six advanced imbalanced approaches on 22 imbalanced real-world datasets, our algorithm has demonstrated superior results in improving the performance of classifiers for minority classes while

maintaining the predicted accuracy for the majority. Moreover, our model has shown great stability over other generative models on training and achieves good results in most scenarios with no adjusting in parameters. Future work will extend our approach to address the imbalance problem for other data types and further reduce the overhead of model computational resources.

REFERENCES

- [1] H. Lin, G. Liu, J. Wu, Y. Zuo, X. Wan, and H. Li, "Fraud detection in dynamic interaction network," *IEEE Trans. Knowl. Data Eng.*, vol. 32, no. 10, pp. 1936–1950, Oct. 2020.
- [2] D. Li, C. Liu, and S. C. Hu, "A learning method for the class imbalance problem with medical data sets," *Comput. Biol. Med.*, vol. 40, no. 5, pp. 509–518, May 2010.
- [3] L. Gong, S. Jiang, L. Bo, L. Jiang, and J. Qian, "A novel class-imbalance learning approach for both within-project and cross-project defect prediction," *IEEE Trans. Rel.*, vol. 69, no. 1, pp. 40–54, Mar. 2020.
- [4] R. Shwartz-Ziv and A. Armon, "Tabular data: Deep learning is not all you need," *Inf. Fusion*, vol. 81, pp. 84–90, May 2022.
- [5] H. He and E. A. Garcia, "Learning from imbalanced data," *IEEE Trans. Knowl. Data Eng.*, vol. 21, no. 9, pp. 1263–1284, Sep. 2009.
- [6] J. Tanha, Y. Abdi, N. Samadi, N. Razzaghi, and M. Asadpour, "Boosting methods for multi-class imbalanced data classification: An experimental review," *J. Big Data*, vol. 7, no. 1, pp. 1–47, Dec. 2020.
- [7] B. Cao, Y. Liu, C. Hou, J. Fan, B. Zheng, and J. Yin, "Expediting the accuracy-improving process of SVMs for class imbalance learning," *IEEE Trans. Knowl. Data Eng.*, vol. 33, no. 11, pp. 3550–3567, Nov. 2021.
- [8] Z. Jan, J. C. Munos, and A. Ali, "A novel method for creating an optimized ensemble classifier by introducing cluster size reduction and diversity," *IEEE Trans. Knowl. Data Eng.*, vol. 34, no. 7, pp. 3072–3081, Jul. 2022.
- [9] N. V. Chawla, D. A. Cieslak, L. O. Hall, and A. Joshi, "Automatically countering imbalance and its empirical relationship to cost," *Data Mining Knowl. Discov.*, vol. 17, no. 2, pp. 225–252, Oct. 2008.
- [10] L. Breiman, "Bagging predictors," *Mach. Learn.*, vol. 24, no. 2, pp. 123–140, Aug. 1996.
- [11] R. E. Schapire et al., "A brief introduction to boosting," in *Proc. Int. Joint Conf. Artif. Intell.*, Jul. 1999, pp. 1401–1406.
- [12] P. Lim, C. K. Goh, and K. C. Tan, "Evolutionary cluster-based synthetic oversampling ensemble (eco-ensemble) for imbalance learning," *IEEE Trans. Cybern.*, vol. 47, no. 9, pp. 2850–2861, Sep. 2017.
- [13] Y. Sun, K. Tang, L. L. Minku, S. Wang, and X. Yao, "Online ensemble learning of data streams with gradually evolved classes," *IEEE Trans. Knowl. Data Eng.*, vol. 28, no. 6, pp. 1532–1545, Jun. 2016.
- [14] Q. Kang, X. Chen, S. Li, and M. Zhou, "A noise-filtered under-sampling scheme for imbalanced classification," *IEEE Trans. Cybern.*, vol. 47, no. 12, pp. 4263–4274, Dec. 2017.
- [15] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic minority over-sampling technique," *J. Artif. Intell. Res.*, vol. 16, pp. 321–357, Jun. 2002.
- [16] H. Han, W.-Y. Wang, and B.-H. Mao, "Borderline-smote: A new over-sampling method in imbalanced data sets learning," in *Proc. Int. Conf. Intell. Comput.*, Aug. 2005, pp. 878–887.
- [17] Y. Xie, M. Qiu, H. Zhang, L. Peng, and Z. Chen, "Gaussian distribution based oversampling for imbalanced data classification," *IEEE Trans. Knowl. Data Eng.*, vol. 34, no. 2, pp. 667–679, Feb. 2022.
- [18] V. García, J. S. Sánchez, and R. A. Mollineda, "On the effectiveness of preprocessing methods when dealing with different levels of class imbalance," *Knowl.-Based Syst.*, vol. 25, no. 1, pp. 13–21, Feb. 2012.
- [19] A. J. Mohammed, M. M. Hassan, and D. H. Kadir, "Improving classification performance for a novel imbalanced medical dataset using smote method," *Int. J. Adv. Trends Comput.*, vol. 9, no. 3, pp. 3161–3172, Jun. 2020.
- [20] R. A. Sowah, M. A. Agebure, G. A. Mills, K. M. Koumadi, and S. Y. Fiawoo, "New cluster undersampling technique for class imbalance learning," *Int. J. Mach. Learn. Comput.*, vol. 6, no. 3, pp. 205–214, Jun. 2016.
- [21] N. A. Azhar, M. S. M. Pozi, A. M. Din, and A. Jatowt, "An investigation of smote based methods for imbalanced datasets with data complexity analysis," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 7, pp. 6651–6672, Jul. 2023.
- [22] D. P. Kingma and M. Welling, "Auto-encoding variational Bayes," in *Proc. Int. Conf. Learn. Represent.*, Apr. 2014.
- [23] I. Goodfellow et al., "Generative adversarial nets," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, Jun. 2014, pp. 2672–2680.
- [24] J. Ho, A. Jain, and P. Abbeel, "Denosing diffusion probabilistic models," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, Dec. 2020, pp. 6840–6851.
- [25] D. Dablain, B. Krawczyk, and N. V. Chawla, "DeepSMOTE: Fusing deep learning and smote for imbalanced data," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 34, no. 9, pp. 6390–6404, Sep. 2023.
- [26] G. Mariani, F. Scheidegger, R. Istrate, C. Bekas, and C. Mallossi, "BAGAN: Data augmentation with balancing GAN," Mar. 2018, *arXiv:2408.00714arXiv:1803.09655*.
- [27] J. Kim et al., "SOS: Score-based oversampling for tabular data," in *Proc. ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, Aug. 2022, pp. 762–772.
- [28] J. Sohl-Dickstein, E. Weiss, N. Maheswaranathan, and S. Ganguli, "Deep unsupervised learning using nonequilibrium thermodynamics," in *Proc. Int. Conf. Mach. Learn.*, Jun. 2015, pp. 2256–2265.
- [29] Y. Song, J. Sohl-Dickstein, D. P. Kingma, A. Kumar, S. Ermon, and B. Poole, "Score-based generative modeling through stochastic differential equations," in *Proc. Int. Conf. Learn. Represent.*, May, 2021, pp. 240–275.
- [30] G. Haixiang, L. Yijing, J. Shang, G. Mingyun, H. Yuanyue, and G. Bing, "Learning from class-imbalanced data: Review of methods and applications," *Expert Syst. Appl.*, vol. 73, pp. 220–239, May 2017.
- [31] G. E. Batista, R. C. Prati, and M. C. Monard, "A study of the behavior of several methods for balancing machine learning training data," *ACM SIGKDD Explorations Newsl.*, vol. 6, no. 1, pp. 20–29, Jun. 2004.
- [32] S. S. Mullick, S. Datta, and S. Das, "Generative adversarial minority oversampling," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, Oct. 2019, pp. 1695–1704.
- [33] Z. Xiao, K. Kreis, and A. Vahdat, "Tackling the generative learning trilemma with denoising diffusion GANs," in *Proc. Int. Conf. Learn. Represent.*, Apr. 2022, pp. 3680–3707.
- [34] M. Arjovsky, S. Chintala, and L. Bottou, "Wasserstein generative adversarial networks," in *Proc. Int. Conf. Mach. Learn.*, Jul. 2017, pp. 214–223.
- [35] A. Radford, L. Metz, and S. Chintala, "Unsupervised representation learning with deep convolutional generative adversarial networks," Nov. 2015, *arXiv:1511.06434*.
- [36] M. Mirza and S. Osindero, "Conditional generative adversarial nets," Nov. 2014, *arXiv:1411.1784*.
- [37] A. Odena, C. Olah, and J. Shlens, "Conditional image synthesis with auxiliary classifier GANs," in *Proc. Int. Conf. Mach. Learn.*, Jul. 2017, pp. 2642–2651.
- [38] J. Engelmann and S. Lessmann, "Conditional wasserstein GAN-based oversampling of tabular data for imbalanced learning," *Expert Syst. Appl.*, vol. 174, Jul. 2021, Art. no. 114582.
- [39] A. Kotelnikov, D. Baranchuk, I. Rubachev, and A. Babenko, "TabDDPM: Modelling tabular data with diffusion models," in *Proc. Int. Conf. Mach. Learn.*, Jul. 2023, pp. 17564–17579.
- [40] L. Xu, M. Skoularidou, A. Cuesta-Infante, and K. Veeramachaneni, "Modeling tabular data using conditional GAN," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, Dec. 2019, pp. 7333–7343.
- [41] J. Kim, C. Lee, and N. Park, "STaSy: Score-based tabular data synthesis," in *Proc. Int. Conf. Learn. Represent.*, May, 2023, pp. 17509–17535.
- [42] Z. Wang, H. Zheng, P. He, W. Chen, and M. Zhou, "Diffusion-GAN: Training GANs with diffusion," in *Proc. Int. Conf. Learn. Represent.*, May, 2023, pp. 34131–34116.
- [43] M. Özbey et al., "Unsupervised medical image translation with adversarial diffusion models," *IEEE Trans. Med. Imag.*, vol. 42, no. 12, pp. 3524–3539, Dec. 2023.
- [44] J. Choi, J. Lee, C. Shin, S. Kim, H. Kim, and S. Yoon, "Perception prioritized training of diffusion models," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, Jun. 2022, pp. 11472–11481.
- [45] K. N. M. Kelly and R. Longjohn, "The UCI machine learning repository," 2007. [Online]. Available: <https://archive.ics.uci.edu>
- [46] L. Breiman, "Random forests," *Mach. Learn.*, vol. 45, no. 1, pp. 5–32, Oct. 2001.
- [47] T. Karras, S. Laine, M. Aittala, J. Hellsten, J. Lehtinen, and T. Aila, "Analyzing and improving the image quality of styleGAN," in *Proc. Conf. Comput. Vis. Pattern Recognit.*, Jun. 2020, pp. 8110–8119.
- [48] G. Lemaître, F. Nogueira, and K. Christos, "Imbalanced-learn: A python toolbox to tackle the curse of imbalanced datasets in machine learning," *J. Mach. Learn. Res.*, vol. 18, no. 17, pp. 1–5, Jan. 2017.
- [49] Y. Sun, L. Cai, B. Liao, and W. Zhu, "Minority sub-region estimation-based oversampling for imbalance learning," *IEEE Trans. Knowl. Data Eng.*, vol. 34, no. 5, pp. 2324–2334, May 2022.

- [50] S. Boughorbel, F. Jarray, and M. El-Anbari, "Optimal classifier for imbalanced data using Matthews correlation coefficient metric," *PLoS One*, vol. 12, no. 6, Jun. 2017, Art. no. e0177678.
- [51] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, Aug. 2016, pp. 785–794.
- [52] T. Cover and P. Hart, "Nearest neighbor pattern classification," *IEEE Trans. Inf. Theory*, vol. TIT-13, no. 1, pp. 21–27, Jan. 1967.
- [53] W.-Y. Loh, "Classification and regression trees," *Interdiscipl. Rev. Data Mining Knowl. Discov.*, vol. 1, no. 1, pp. 14–23, Jan. 2011.
- [54] D. W. Hosmer Jr., S. Lemeshow, and R. X. Sturdivant, *Applied Logistic Regression*, vol. 398. Hoboken, NJ, USA: Wiley, Apr. 2013.
- [55] Y. Zhang, B. Kang, B. Hooi, S. Yan, and J. Feng, "Deep long-tailed learning: A survey," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 45, no. 9, pp. 10795–10816, Sep. 2023.
- [56] J. Demšar, "Statistical comparisons of classifiers over multiple data sets," *J. Mach. Learn. Res.*, vol. 7, pp. 1–30, Jan. 2006.
- [57] J. Ho and T. Salimans, "Classifier-free diffusion guidance," Jul. 2022, *arXiv:2207.12598*.



Shiqi Ren received the BEng degree in automation in 2022 from Northeastern University, Shenyang, China, where he is currently working toward the PhD degree in control theory and control engineering with the State Key Laboratory of Synthetical Automation for Process Industries. His research interests include imbalanced learning, deep generative models, anomaly detection, and their applications in complex industrial processes.



Jinliang Ding (Senior Member, IEEE) received the bachelor's, master's, and PhD degrees in control theory and control engineering from Northeastern University, Shenyang, China, in 2001, 2004, and 2012, respectively. He is currently a professor with the State Key Laboratory of Synthetical Automation for Process Industry, Northeastern University. He has authored or co-authored more than 200 refereed journal papers and refereed papers at international conferences. He has also invented or co-invented more than 50 patents. His research interests include

modeling, plant-wide control, and optimization for the complex industrial systems, machine learning, industrial artificial intelligence, and computational intelligence and application. Dr. Ding was a recipient of the Young Scholars Science and Technology Award of China in 2016, the National Science Fund for Distinguished Young Scholars in 2015, the National Technological Invention Award in 2013, and three First Prizes of Science and Technology Awards of the Ministry of Education in 2006, 2012, and 2018, respectively. One of his articles published on Control Engineering Practice was selected for the Best Paper Award from 2011 to 2013. He was an associate editor for *IEEE Transactions on Evolutionary Computation*, *IEEE Transactions on Emerging Topics in Computational Intelligence*, and *IEEE Transactions on Circuits and Systems II: Express Briefs*.



Yiu-ming Cheung (Fellow, IEEE) received the PhD degree from the Department of Computer Science and Engineering, The Chinese University of Hong Kong, Hong Kong. He is currently a chair professor (Artificial Intelligence) with the Department of Computer Science, HongKong Baptist University, Hong Kong, SAR, China. His research interests include machine learning, visual computing, data science, pattern recognition, multi-objective optimization, and information security. He is also an editor-in-chief of *IEEE Transactions on Emerging Topics in Computational Intelligence*. He was an associate editor for *IEEE Transactions on Cybernetics*, *IEEE Transactions on Cognitive and Developmental Systems*, *IEEE Transactions on Neural Networks and Learning Systems* from 2014 to 2020, *Pattern Recognition*, *Pattern Recognition Letters*, and *Neurocomputing*. He is a Fellow of AAAS, IET, BCS, and AAIA. For details, please refer to: <https://www.comp.hkbu.edu.hk/~ymc>.